



Project Report

Intern Name: Ahmed Islam

Department: Artificial Intelligence

Organization: IR Solutions

Designation: AI Intern

Week # 10

Submitted to: Asad Ahmed

Submission Date: 12-Sep-2025

Table of Contents

Project Report	1
Theoretical Learning.....	3
1. Introduction to LangGraph	3
Key differences between LangGraph and LangChain:	3
Why use LangGraph:.....	3
2. Foundations of LangGraph	4
Core principles.....	4
3. Agentic AI Architecture	4
4. Human-in-the-Loop (HITL) Integration.....	5
Practical Implementations.....	5
Project 1: AI Research Assistant Agent (LangGraph).....	5
Implementation Details	5
Key Features	6
Project 2: Email Extraction from Website Domains Manually & also tried ChatGpt Api implementation	6
Challenges Faced	7
Conclusion.....	7

Theoretical Learning

This week, I explored Agentic AI systems, their architecture, and how LangGraph can be used to build these systems. I also attempted to apply these concepts to real-world use cases, like creating a Research Assistant Agent and performing email extraction from domains manual coding and also by using the Chatgpt Api key.

Below is the detailed breakdown of what I learned:

1. Introduction to LangGraph

LangGraph is a framework designed for building stateful, agentic AI systems that require complex decision-making, branching logic, and memory.

While LangChain is excellent for creating linear LLM pipelines (Chains, Tools, Memory, Agents), it can become complicated when building multi-step, reactive, or autonomous agents. This is where LangGraph becomes useful.

Key differences between LangGraph and LangChain:

Aspect	LangChain	LangGraph
Architecture	Pipeline (linear flow)	Graph-based (state machine flow)
Use Case	Simple LLM chains, RAG systems	Complex autonomous agents
Memory Handling	Basic (ConversationBuffer, VectorStore memory)	Advanced persistent state across nodes
Control	Implicit	Explicit control of agent state transitions

Why use LangGraph:

- It allows creating **custom state graphs** where each node represents an action, and edges represent transitions.
- Enables **conditional branching** and **parallel task execution**.
- Makes it easier to visualize and debug the flow of agent reasoning.

2. Foundations of LangGraph

LangGraph works by representing the logic of an AI agent as a **graph of nodes**. Each node can represent:

- A model call
- A tool usage
- A reasoning step
- A memory update

Core principles

- **Nodes:** Functions or tasks to execute (e.g., “search”, “summarize”, “answer”).
- **Edges:** Define how the system moves between nodes based on conditions.
- **State:** Shared data passed between nodes that represent the current memory/context of the agent.
- **Persistence:** The graph can maintain state over long-running interactions, unlike stateless chains.

This design makes LangGraph suitable for **Agentic AI**, which requires statefulness, context retention, and adaptive reasoning.

3. Agentic AI Architecture

Agentic AI systems are designed to **autonomously plan, reason, and act**. They consist of the following main components:

- **Brain**
The central reasoning unit, usually an LLM (e.g., GPT, Claude, Gemini). It makes decisions based on context and goals.
- **Orchestrator**
The controller that decides **which tool or node the system should execute next**. It's like a scheduler that plans the agent's next step.
- **Tools**
External capabilities the agent can use (APIs, calculators, web search modules, databases, scrapers, etc.).

- **Memory**

A storage system to remember previous interactions, results, and user context. In LangGraph, memory can persist across multiple runs, allowing the agent to build long-term knowledge.

- **Supervisor**

A higher-level component that monitors and controls the agent's behavior, ensuring it stays aligned with rules, ethics, and task objectives.

These components work together to enable **autonomous and multi-step reasoning**.

4. Human-in-the-Loop (HITL) Integration

I also studied **Human-in-the-Loop (HITL)** systems, which combine **human judgment with AI decision-making**.

- Humans can **review, validate, or override** decisions made by the agent.
- This is crucial when:
 - Tasks are high-stakes or sensitive.
 - LLM outputs need validation for correctness.
- LangGraph supports **inserting human approval steps** in the graph workflow.

This allows building safer and more reliable AI systems where human feedback helps correct or guide the agent.

Practical Implementations

This week, I built **two main projects** to apply the above concepts.

Project 1: AI Research Assistant Agent (LangGraph)

Goal: Build a multi-step AI agent that can help with research tasks (summarizing articles, extracting key points, answering user questions) using LangGraph.

Implementation Details

- Created a **LangGraph-based state graph** where:
 - One node fetched content from a source.

- Another node summarized the content.
 - Another node answered user questions.
- Integrated **Memory** so the assistant could recall previous questions in the same session.
- Included a **Human-in-the-Loop step** where I could approve or reject the summaries before they were finalized.

Key Features

- Used **LLM as the Brain** to do reasoning.
- Used **custom Python tools** as Tools nodes.
- Used **LangGraph Orchestrator** to route between steps.
- Used **Memory** to store conversation history for contextual responses.

This project helped me understand how to design **Agentic AI systems** from scratch using LangGraph.

Project 2: Email Extraction from Website Domains Manually & also tried ChatGpt Api implementation

Goal: Extract email addresses from company domain names.

Steps Taken:

- Collected a list of website domains from an Excel file.
- Wrote a **web scraping script** to extract email addresses from those domains.

Challenges Faced:

- Most websites **do not list emails publicly** (they hide them behind contact forms or obfuscate them).
- Many domains returned **None** because of missing visible emails.
- Tried using **ChatGPT API** and **Gemini API** to infer emails, but:
 - These models **do not have direct web search capability via API**.
 - They cannot access live websites, so they couldn't fetch emails by domain name.

Result:

- This approach not worked because we have a token limit for the whole website content and also APIs don't have a real time web search capability.
- Could not extract emails for the majority of domains.

Challenges Faced

1. LLM API Limitations

- ChatGPT and Gemini APIs **cannot access the internet**, so they can't perform live email lookups.
- They rely on their training data and do not have browsing capabilities through the API.

2. Data Availability Issue

- Many websites **do not publish their emails** on pages, so scraping returns nothing.

3. Infrastructure Constraints

- Our Laptops are **not capable of handling large-scale computational tasks** like heavy scraping.
- Web scraping at scale requires **robust servers, concurrency handling, proxies, and error recovery**, which are not available in our setup right now.

These challenges highlight the gap between **LLM capabilities** and **real-time data retrieval tasks**.

Conclusion

This week gave me **hands-on exposure to Agentic AI systems** and the **LangGraph framework**, helping me understand how to build **autonomous, memory-powered agents** with human oversight.

I also learned about the **real-world limitations of LLM APIs** when it comes to tasks like **email extraction**, where **live data access** is required.